Características de un DBMS

Introducción

El uso de Bases de Datos es tan importante que podríamos decir que no existe prácticamente empresa (de negocios, educacional, gobierno, etc.) que no posea por lo menos una. En este apunte citaremos sus características y resaltaremos sus principales diferencias con un sistema tradicional de procesamiento de archivos.

Además, toda organización precisa manejar información: que surge en papel (ejemplo: factura recibida, libros de una biblioteca), que se emite en papel (ejemplo: factura emitida), que nace de un archivo electrónico (Ejemplo: documento de word, archivo que llega por mail), que nace en cinta (Ejemplo: un video de capacitación), que se escucha (ejemplo: mensaje que nos dejaron en una casilla de teléfono, conversación telefónica), etc. A continuación analizaremos el rol que ocupan las bases de datos en los sistemas de información

1. Sistemas de Información

La información que maneja una organización es voluminosa y heterogénea (viene en distintos formatos: papel, audio, video). Los **sistemas de información** se encargan de manipular dicha información y son usados por las organizaciones con el objetivo de procesarla, pudiendo así consultarla rápidamente y sacar ventaja en la toma de decisiones, característico de cualquier organización competitiva. Tan complejo resulta manejar semejantes volúmenes de información heterogénea que cada vez es mayor la tecnología emergente que van incorporando las empresas con el fin de permitir que dicha información sea realmente utilizada, en vez de archivada y olvidada.

La información sólo puede usarse en la medida en que se encuentre pre-procesada. Por ejemplo, si una biblioteca adquiere libros y arma un catálogo para su consulta incluyendo sólo su autor y editorial, es probable que sea aprovechada por poca gente, ya que sólo los que tengan conocimiento **sobre qué escribe determinado autor** estarían consultando sus libros. Si en cambio se incluye su título, resulta más relevante porque el mismo suele incluir palabras que informan sobre el contenido del libro. Si pudiera, además incluirse el prefacio o el índice sería más útil aún porque a veces el título no indica demasiado: “Las Perlas de Bentley” es un libro sobre trucos de programación, pero su nombre no sugiere mucho sobre su contenido. Lo mismo ocurre con el nombre de un archivo que no siempre sugiere lo que contiene.

Las **bases de datos** son sólo una parte de los sistemas de información.

Silvia Alicia Gómez & Leticia Irene Gómez

2. *Bases de Datos y Sistema de Manejo de Base de Datos (DBMS)*

Base de Datos

Colección de datos relacionados, que cumple con las siguientes propiedades:

- Representa algún aspecto del mundo real (provienen de alguna fuente o mini-universo)
- Tiene coherencia lógica con algún significado inherente (no son números aleatorios)
- Se diseña, construye y se completa con datos para un propósito específico. Puede tener diferentes grupos de usuarios cada uno con intereses distintos y además cuenta con ciertas aplicaciones preconcebidas para lograr cierto objetivo.

Una Base de Datos ofrece una abstracción sobre los datos que almacena, por cuanto oculta detalles de su almacenamiento y manejo. Además no posee límite en la cantidad de información que puede manipular. Una planilla Excel no es una base de datos porque no cumple con la tercera propiedad (no se pueden definir perfiles de usuarios que accedan a determinada parte de la planilla o tengan distintos tipos de permisos) y, aunque pueda almacenar datos, si el volumen es muy grande el recupero de la información en forma eficiente se vuelve conflictivo.

Sistema de Manejo de Base de Datos
(Sistema de Gestión de Base de Datos, Database Management System o DBMS)

Es una colección de programas que permiten **crear** y **mantener** una Base de Batos.

Su **creación** implica definir los tipos de datos, estructura y restricciones de los datos que se almacenen. **Mantenerla** implica actualizar sus datos para que reflejen los cambios del mini-universo que representa, generar reportes y poder consultar la información que posee almacenada.

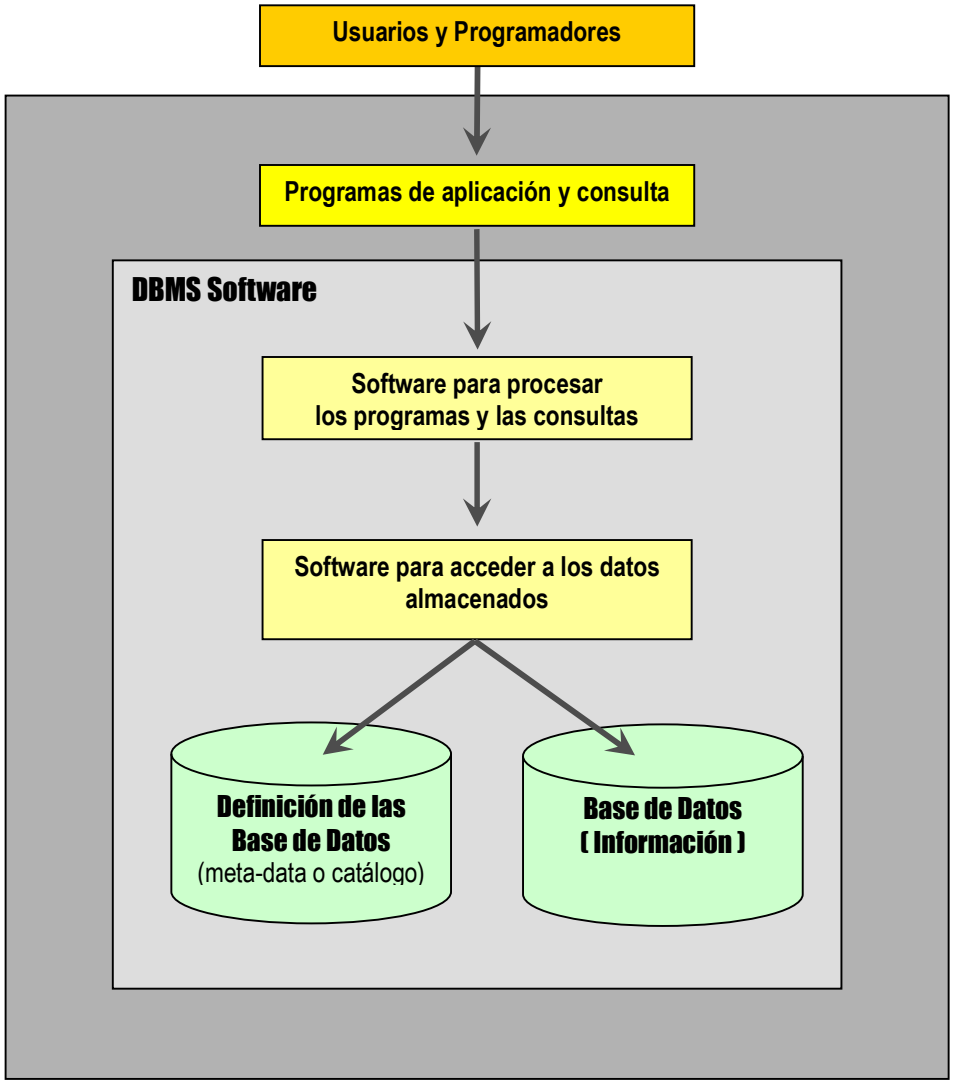
El software de un DBMS no responde a fines específicos: puede ser usado para educación, gobierno, medicina, etc. Cada Base de Datos manejada por un DBMS sí tiene un objetivo específico.

Lo primero que podríamos pensar para desarrollar un DBMS es diseñar un sistema de archivos para almacenar la información y un conjunto de programas para poder crear y mantener la base de datos, tal cual lo citamos previamente.

Obviamente todo podría ser desarrollado, por ejemplo en lenguaje ANSI C. Los archivos que guardan la información podrían estar formados por *structs*, y las aplicaciones que permitan manipular los datos podrían tener árboles (AVL, B, etc.) o usar técnicas de Hashing para poder responder en forma eficiente a las consultas para lo cual la base fue diseñada. En la próxima sección discutiremos en profundidad cuáles son las características que deben implementarse en los DBMS.

En el siguiente gráfico resumimos el esquema general:

Silvia Alicia Gómez & Leticia Irene Gómez



3. *Diferencias entre una DMBS y un Sistema de Procesamiento de Archivos*

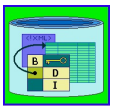
➢ **Autodescripción (Catálogo)**

Como se observa en la figura anterior, un DBMS no sólo contiene los datos de la base sino la completa definición de la misma, la cual se guarda en el *catálogo (catalog)*. Dicho *catálogo* posee la estructura subyacente, el formato y tipo de cada ítem, y restricciones de los datos (si las hubiera).

En general el *catálogo* es usado por el software DBMS, pero podría ser consultado por los usuarios que desearan conocer la estructura subyacente.

Un software DBMS no se construye para un fin específico, puede ser usado por una empresa de informática, un centro de medicina, un banco, una universidad, etc. En un sistema tradicional de archivos la definición de los datos es parte de los programas de aplicación: un programa en C guarda los *structs*, y después un programa en Pascal tendría dificultades para levantarlos (alineamientos). No se almacena la estructura de datos como parte de la especificación. Con un software DBMS se pueden manejar distintas bases de datos porque la información se extrae del catálogo.

Silvia Alicia Gómez & Leticia Irene Gómez



➢ **Independencia entre datos y programas**

Una base de datos separa el programa de los datos, ofreciendo un nivel de abstracción superior. Como en un sistema tradicional de archivos las estructuras de los archivos (por ejemplo, *structs*) están reflejados en los programas que los manipulan, un cambio en la estructura requiere un cambio en los programas que acceden a dicho archivo. Ejemplo: si representamos a los empleados por su legajo y sueldo, y después deseamos caracterizarlos también por su antigüedad, deberíamos cambiar la estructura y los programas que accedan a la misma. Con el uso de TAD se simplifica el caso, porque sólo se ven afectados los programas que constituyen la implementación del TAD. Como la estructura de una Base de Datos se almacena en el catálogo, tenemos independencia entre programa y datos. Así es como un DBMS ofrece una abstracción de los datos.

Los diseñadores de Bases de Datos deben representar los datos con distintas estructuras muy complejas para poder llevar a cabo el objetivo de poder ofrecer un recupero de la información eficiente. En un sistema de archivos tradicional las aplicaciones que puedan manejar consultas en forma eficiente se vuelven muy complejas.

Además como en un sistema de archivos sólo los programas pueden especificar la lógica, cada vez que se desee realizar una consulta diferente habría que hacer un nuevo programa. En un DBMS se posee un lenguaje potente para poder realizar consultas en forma sencilla. Ejemplo: listar aquellos empleados cuya edad promedio supere la media, ordenados por fecha de ingreso.

➢ **Ofrece vistas a distintos grupos de usuarios (views)**

La idea de no permitir acceder a todos los usuarios de la misma manera y sólo autorizar a ver la porción de datos que les incumbe es un objetivo importante. Esto se logra por medio de vistas (views) que permiten que cada usuario de la base de datos acceda sólo desde su perspectiva, pudiendo ésta ser un subconjunto de la base, o inclusive pudiendo contener datos derivados (virtuales, no almacenados). Ejemplo: la liquidación neta del sueldo de los empleados puede no estar guardada en la base de datos, sino deducirse en base a otros datos.

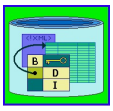
Dar permisos a cierta zona de un archivo no es fácil de manejar desde un sistema convencional de archivos, ni aún con ayuda de un buen sistema operativo. Por ejemplo: no basta con darle permisos a un archivo para que lo vean los usuarios del Departamento de Recursos Humanos pero no lo puedan acceder los de Facturación, porque también hay que hacer que algunos datos que corresponden a los gerentes no puedan ser vistos ni siguiera por el área de Recursos Humanos.

➢ **Integridad de los Datos**

Al no poder establecer reglas entre los archivos de un sistema puede resultar difícil verificar la consistencia de la información almacenada en los distintos archivos. El problema es que todas las validaciones provienen de los programas que acceden a los archivos y un cambio de criterio u omisión en cierta aplicación puede provocar un error grave de inconsistencia de información.

En un DBMS resulta más sencillo porque se pueden establecer restricciones en el momento en que se crea la base de datos. Ejemplo: se puede estipular el formato de los datos para que no se inserte un dato con distinto formato (típico de los números telefónicos), o pedir que no se inserte un dato si el mismo no supera la media alcanzada hasta el momento, etc.

Silvia Alicia Gómez & Leticia Irene Gómez



➢ **Manejo de la Concurrencia**

Resulta complicado coordinar el acceso a un sistema de archivos desde aplicaciones simultáneas. Ejemplo: si dos usuarios desde distintas computadora intentaran hacer la reserva de un cierto asiento a una sala de teatro, los programas comienzan consultando el archivo que posee los asientos con un 1 o un cero, según esté vendido o disponible respectivamente, ambos levantan el valor 0 para cierto asiento (creyendo que está vacío), luego de esto una de las aplicaciones coloca un 1 (para indicar que se vendió) y un instante después la otra aplicación coloca un 1 encima (para indicar que se vendió), ambos poseen la misma entrada sin saberlo.

Un DBMS maneja el control de la concurrencia, para asegurar que no se produzcan inconsistencias de datos por múltiples actualizaciones no sincronizadas. Su rol es asegurar que las transacciones concurrentes operen correctamente sin interferencia.

➢ **Procesamiento Transaccional (atomicidad)**

Un sistema de computación está sujeto a fallas de hardware y software. Se debe poder garantizar que un conjunto de operaciones sea considerado atómico: si se produce un error en el medio de la ejecución de las operaciones hay que deshacer todo lo hecho antes de comenzar con el conjunto atómico. Ejemplo: se desea transferir un cierto monto entre dos cuentas bancarias, la primera operación consiste en deducir el monto de la primera cuenta, y la segunda en sumarlo al saldo de la segunda. Si después de realizar la primera operación se produce un error (alguien patea el cable de la computadora y se desconecta) como el conjunto de ambas operaciones se consideraba atómica, al fallar la segunda se deben deshacer automáticamente los efectos producidos por la primera.

En un sistema de archivos asegurar el comportamiento atómico es bastante complicado.

Las últimas dos características son las más importantes que distinguen a un sistema de procesamiento de archivos de un DBMS.

Todo lo citado anteriormente es tan común y necesario que resulta absurdo malgastar semejante esfuerzo de programación cada vez que se quiera manejar información. Por tal motivo surgieron los desarrollos de DBMS que proporcionan todas las características, desvinculando a los programadores de complicar sus aplicaciones con detalles innecesarios.

Los desarrolladores de DBMS manejan todas estas cuestiones y los usuarios de las mismas toman ventajas de todas ellas. Los desarrolladores de DBMS, igual que los que escriben Sistemas Operativos o Compiladores realizan Software de Base. Pretender que cada vez que un programador (o empresa) desee manipular información compleja escriba su propia DBMS resulta tan absurdo como pretender que todo programador (o empresa) escriba sus propios Sistemas Operativos o Compiladores.

Silvia Alicia Gómez & Leticia Irene Gómez

4. *Modelo de Datos*

La idea de separar los detalles del almacenamiento físico de los datos se logra por medio de una abstracción. Precisamente un modelo de algo existente en la realidad (sea tangible o conceptual) permite tomar lo representativo de ese algo y dejar de lado lo irrelevante. En el caso de DBMS podemos definir:

Modelo de Datos

Conjunto de conceptos que se utilizan para describir la estructura de la base de datos (tipos de datos, relaciones y ciertas restricciones).

La mayoría de los modelos de datos ofrecen también un conjunto de operaciones básicas (built-in) para el recupero y actualización de la base de datos, e inclusive algunas permiten especificar comportamiento o sea operaciones definidas por el usuario.

Existen distintas categorías de modelos de datos, según el nivel de abstracción que poseen para los datos:

- **Conceptual, de alto nivel o semántico**: provee conceptos que tiene que ver con la forma con que los usuarios perciben los datos. Pueden ser:
  - **Basados en entidades**. Utilizan los siguientes conceptos: entidad o *entit*y para representar un concepto del mundo real (Ejemplo: un alumno, una materia), atributo o *attribut*e para representar una propiedad interesante de una entidad (Ejemplo: el nombre de un alumno), y relación o *relationship* para representar una interacción entre dos o más entidades (Ejemplo: la relación cursa entre la entidad alumno y la entidad materia). En esta materia vamos a estudiar el **modelo entidad-relación**.
  - **Basados en objetos** (fuera del alcance de esta materia)

- **De representación, lógico, implementación u orientado a registro**: representan los datos usando estructuras de registro. Los más importantes comercialmente son:
  - **Modelo de Red (Network Model)**: En la década del ‘60 se forma la base para este modelo debido a la conferencia sobre Data Systems Languages Database Task Group (CODASYL DBTG). Por los ‘70s se arman los sistemas comerciales que siguieron la propuesta CODASYL DBGT: IDMS System, DMS 1100 (Unisys), IMAGE (HP), VAX-DBMS (Digital).
  - **Modelo Jerárquico (Hierarchical Model)**: Implementado a mediados de la década del ’60. Surge la familia de los sistemas Information Management System de IBM (IMS).
  - **Modelo Relacional (Relational Model)**: Propuesto por Codd, investigador de IBM, en 1970. El primer sistema comercial surge en 1981-1982. Hoy en día es el más común: Oracle, Sybase, Infomix, DB2, SQL Server, PostgreSQL, MySql, etc.

- **Físico, de bajo nivel o interno**: provee conceptos que describen los detalles de cómo los datos se almacenan en la computadora: formato, ordenamiento y la estructura de acceso para lograr eficiencia.

Silvia Alicia Gómez & Leticia Irene Gómez

5. *Meta-Data (intensión o definición) y Datos (extensión)*

Ya citamos como una característica importante que debe poseer un DBMS la necesidad de **autodescripción**. Cualquiera sea el modelo de datos, hay que distinguir entre la **descripción de la base de datos** (esquema de la base de datos, meta-data, catálogo, database schema, intensión o *catalog*) y los **datos de la misma** (extensión).

El esquema de la base de datos suele tener un comportamiento bastante estático, o sea no suele cambiar con frecuencia en el tiempo. Se lo especifica por medio de un diseño.

Los datos, en cambio, son los que mantienen relación con el universo de discurso, y por lo tanto cambian frecuentemente por medio de las actualizaciones precisas.

Ejemplo:

Para el caso del alumno que cursa ciertas materias el esquema que nos permite representar esa porción de la realidad siempre es el mismo y podría estar dado por el siguiente diagrama:

ALUMNO

Legajo	Nombre

MATERIA

Codigo	Nombre

INSCRIPTO

Legajo	Codigo

En cambio, el estado de la base de datos cambia instante a instante. En el ejemplo dado, en cada cuatrimestre se eliminan las inscripciones anteriores y aparecen otras, de un año para otro aparecen nuevos alumnos y se eliminan los que egresan, cambian los nombres de las materias que se dictan, etc.).

Cuando se diseña una base de datos se debe proveer dicho **esquema** al DBMS. Luego, los datos empiezan en un **estado vacío** el cual va cambiando en el tiempo. El DBMS es **responsable de garantizar que todos los estados de las bases de datos sean válidos**, o sea si satisfacen la estructura y las restricciones especificadas en el esquema.

Ejemplo:

Si el esquema para los alumnos que se inscriben especifica la restricción de que "no se puede permitir que un alumno curse una materia que no exista", el DBMS debe verificar cada vez que actualice la base de datos (inserción o modificación de registros en INSCRIPTO) que el Codigo indicado se corresponda con algún registro en MATERIA.

Silvia Alicia Gómez & Leticia Irene Gómez

Muy Importante

La **especificación de un esquema correcto** es muy importante ya que libera a las aplicaciones (programas) del control de ciertos errores. Los estados válidos son garantizados por el DBMS utilizando dicho esquema. Así es como tenemos otra vez la importancia de la independencia entre datos y programas que ya mencionamos como una característica de los DBMS.

El esquema proporcionado es guardado por el DBMS en el ***catálogo***.

6. Arquitectura de Tres Niveles

En secciones anteriores hemos mencionado tres características que poseen los DBMS: **autodescripción** (el catálogo o esquema almacena la descripción de la base de datos), **separación de programas y los datos**, soporte de **vistas para usuarios** (views).

Para estandarizar y garantizar estos objetivos el comité ANSI/SPARC Study Group on Database Management Systems propuso una arquitectura de tres niveles (three-schema arquitecture) que deben tener los DBMS:

- **Nivel Interno o Vista Interna (Internal Level o Internal View)**: posee un esquema interno que describe la estructura de almacenamiento físico de la base de datos. Este esquema usa un modelo físico de los datos y describe los detalles completos del almacenamiento de los datos y acceso a la base de datos. Ejemplo: describe como la información descripta en el esquema conceptual se representa físicamente para proveer la mejor *performance*, así internamente se usa un árbol B+ para la consulta por edad o bien una tabla de *hashing* para los nombres.

- **Nivel Conceptual o Vista Conceptual (Conceptual Level o Conceptual View)**: posee un esquema conceptual que describe la estructura de la base de datos completa para la comunidad de usuarios. Obviamente ofrece un nivel de abstracción, ocultando los detalles físicos de almacenamiento. Describe la semántica de las entidades, tipos de datos, relaciones, operaciones de usuario y restricciones. Ejemplo: indica que se precisa representar a la entidad empleado caracterizándola por su nombre formado de 30 caracteres, su dirección de 20 caracteres y su edad entera y comprendida entre 18 y 70.

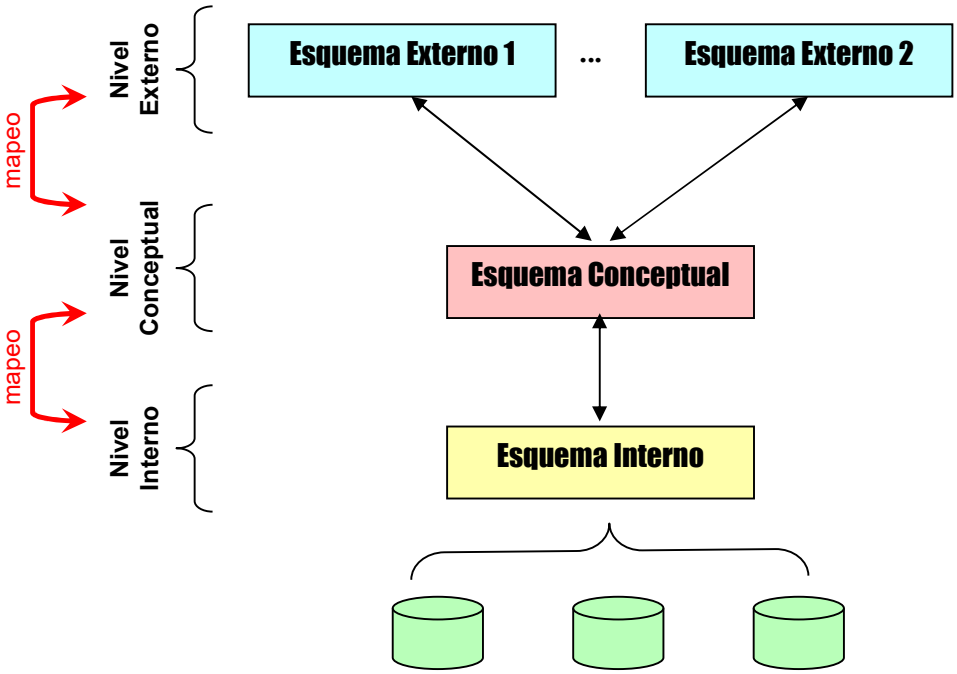
- **Nivel Externo o Vista Externa (External Level o External View)**: posee varios esquemas externos o vistas de usuarios. Cada uno de ellos describe parte de la base de datos para un grupo de usuarios en particular (ocultando el resto de la base de datos). En general la comunidad de usuarios que están interesados en la base de datos sólo lo está en una fracción de la misma. Ejemplo: los usuarios del grupo que manejan los sueldos de los empleados de una empresa no tienen por qué ver los datos relativos al legajo de personal de los empleados, y los usuarios del grupo de los vendedores de la misma no tienen por qué ver los sueldos de sus compañeros.

Importante

Los tres esquemas son descripciones de los datos, y los datos sólo existen en realidad en el nivel físico.

Los Sistemas de Manejo de Base de Datos deben realizar un mapeo, o sea una transformación entre los niveles según corresponda.

Silvia Alicia Gómez & Leticia Irene Gómez



Es muy importante entender el motivo de esta arquitectura: su separación permite el cambio de un esquema en un nivel sin afectar lo esquemas superiores. Esto se logra gracias a la independencia física y lógica de los datos. Claramente pueden verse que la capacidad de cambiar el esquema interno sin tener que alterar los esquemas conceptuales o externos se logra gracias a la **independencia física de los datos**. Ejemplo: si después de un tiempo el diseñador de una base de datos se da cuenta que determinada consulta está demorando demasiado su respuesta, puede agregar una estructura adicional sobre el campo consultado para mejorar performance sin afectar al resto de los esquemas ni aplicaciones.

A su vez un cambio en el esquema conceptual no debe afectar al esquema exterior debido a la **independencia lógica de los datos**. Ejemplo: agregar una restricción no debería afectar a las vistas ni aplicaciones, ni tampoco el separar los datos de una sola entidad entre varias.

Aclaración

Si se respeta esta arquitectura de tres niveles, un cambio en un esquema no interfiere con el resto de los esquemas. Lo que sí cambia el DBMS es el mapeo entre un esquema y otro.

Silvia Alicia Gómez & Leticia Irene Gómez

7. Perfiles de las Personas que interactúan con un DBMS

Los DBMS son accedidos con distintos roles por distintas personas. Citaremos un primer grupo correspondiente a los que proveen software para aquellos que deseen adquirir algún DBMS; y un segundo grupo presente en cualquier empresa que ha adquirido algún DBMS.

Primer Grupo:

- ***Implementador de una DBMS:*** es la persona que está involucrada en el proyecto de desarrollar un DBMS. Se las tiene que ver con todas las características antes citadas: garantizar seguridad, balancear la carga de procesar una compleja consulta entre distintos procesadores, ofrecer excelente performance. Ejemplo: implementador de DB2 Universal Enterprise Server 8.5 de IBM.

- ***Desarrollador de Herramienta para DBMS:*** suele ser una persona que trabaja en una empresa independiente a la que provee DBMS pero quiere desarrollar tecnología que funcione con distintas DBMS para permitir diseñar una base de datos, monitorear performance, desarrollar interfaces gráficas que interactúen con bases de datos, etc. Ejemplo: empresa que desarrolle una herramienta gráfica que automáticamente genere la especificación del esquema para distintas DBMS, como por ejemplo Informix, Oracle, DB2, etc.

Segundo Grupo:

- ***Administrador:*** llamado DBA (Database Administrator) es la persona encargada de autorizar los accesos a las bases de datos, monitorear su uso (para detectar intrusos y ver la performance de la misma) y administrar los recursos de software y hardware necesarios para el buen funcionamiento (hacer backups, etc).

- ***Diseñador:*** es el responsable de identificar la estructura a representar, indicar las restricciones que deben cumplirse. Sus tareas surgen de los requerimientos que le solicitan los usuarios de la base de datos. También desarrolla las vistas según los grupos de usuarios que accederán a la misma.

- ***Usuarios:*** Son las personas que consultan y actualizan la información de la misma. Existen a su vez distintos tipos de usuarios finales:
  - **Comunes**: usan aplicaciones ya hechas y acceden a la base de datos automáticamente (quizás sin tener conocimiento de informática). Ejemplo: un empleado que vende entradas para un teatro.
  - **Programadores**: que realizan aplicaciones que embeben consultas a base de datos o escriben programas en 4GL para ellos. Desarrollan las aplicaciones que usan los usuarios comunes.
  - **Ocasionales**: eventualmente realizan alguna consulta imprevista a la base de datos. Ejemplo: un gerente que desee encontrar, quizás por única vez, la relación entre cierta información almacenada. Obviamente precisa conocer el lenguaje de consulta necesario.

Aclaración

En esta materia vamos a desarrollar los roles de **Diseñador** y **Usuarios**.

Silvia Alicia Gómez & Leticia Irene Gómez

8. Lenguajes para una Base de Datos

Una vez que se diseñó determinada base de datos se debe definir los 3 niveles definidos por el `ANSI/SPARC Study Group`. La especificación del esquema conceptual se lo hace con un lenguaje llamado Lenguaje de Definición de Datos (Data Definition Language o DDL). El DBMS posee un compilador de DDL que traduce la descripción almacenándola en el catálogo de la misma. La especificación del esquema interno se lo hace con el Lenguaje de Definición de Almacenamiento (Storage Definition Language o SDL). Finalmente se diseñan las vistas por grupos de usuarios con el Lenguaje de Definición de Vistas (View Definition Language o VDL). Cuando se quiere manipular la información que mantiene la base de datos (insertando, eliminando o actualizando datos) se lo hace con el lenguaje de Manipulación de Datos (Data Manipulation Language o DML). Hoy en día todos esos lenguajes están unificados en uno solo: SQL (Structured Query Language).

Cuando una sentencia de un DML se la usa en forma interactiva desde una terminal recibe el nombre de **query language**. Dicha sentencia es tomada por una parte del DBMS llamado **query compiler** que parsea y analiza el requerimiento y genera invocaciones al run-time database processor (proceso que manipula los accesos a la base de datos).

Cuando una sentencia de un DML se encuentra embebida en un lenguaje de programación (ejemplo: C) el que embebe se denomina **host language** y el DML se llama **data language.** En este caso el módulo del DBMS llamado **precompiler** analiza las sentencias del programa fuente: las que corresponden a sentencias DML se las pasa al **DML compiler**, y el resto se las pasa al **Host Language compiler**. La salida de ambos se linkedita y dicho código ejecutable es el que se utiliza. En este caso dicho código posee ciertas invocaciones al run-time database processor.

Existe otra forma de embeber invocaciones a una base de datos en un lenguaje de programación: por medio de la especificación ODBC (Open Database Connectivity) que ofrece un API (application programming interface) que permite que las aplicaciones accedan a múltiples DBMS. Una aplicación ya compilada que use ODBC puede conectarse una vez a cierto DBMS y otra vez a otro DBMS (obviamente con la mismas características) sin necesidad de recompilar, porque el API de ODBC posee funciones genéricas que en tiempo de ejecución invocan a las específicas del driver del DBMS correspondiente (o sea hay un mapeo entre las funciones embebidas en el programa y el driver con que decido conectarme) que es el que invoca al run-time processor en cuestión. Obviamente esta ventaja de tener código ya ejecutable que se conecta dinámicamente a cualquier DBMS se logra a expensas de performance (la traducción de una invocación genérica en una concreta los hace el driver específico y en tiempo de ejecución).

En general las interfaces para manipulación de bases de datos que se ofrecen son variadas: se puede usar directamente algún editor que venga con el DBMS para consultas aisladas, se puede desarrollar rápidamente una aplicación 4GL, se puede desarrollar alguna aplicación con un lenguaje como C que tenga la posibilidad de embeber accesos a las bases de datos, o se puede usar alguna herramienta generadora de aplicativos con acceso a bases de datos (que no vienen con la base de datos, como es el caso de PowerBuilder).

Silvia Alicia Gómez & Leticia Irene Gómez

9. Arquitectura Cliente/Servidor - Bases de Datos Distribuidas

Como se mencionó previamente, un DBMS debe permitir acceder concurrentemente a múltiples usuarios. Esto significa que el DBMS reside en una/varias computadora/s y los clientes en otras.

Así es como un DBMS funciona en una red de computadoras, pudiendo manejarse:

- **En una arquitectura cliente/servidor centralizada**: el software de DBMS y las bases de datos residen en una misma computadora y las aplicaciones clientes en otras computadoras. Los clientes realizan requerimientos que son enviados por la red hacia el servidor, el cual se encarga de procesar lo pedido, enviando la respuesta al cliente. Se llama también arquitectura client/server 2-tier (de dos niveles: el front-end está en los clientes, y el back-end en el servidor).

- **En forma distribuida**: el software DBMS y las bases de datos residiendo en distintas computadoras y comunicándose entre ellas con el fin de atender los requerimientos de las aplicaciones clientes (que a su vez están en otras computadoras). Hoy en día se disponen de DBMS distribuidas **homogéneas** y **heterogéneas**. Las primeras poseen distribuido el mismo software de DBMS con el fin de para balancear la carga debido a un problema de escalamiento. La segunda comunica distintos software de DBMS (e inclusive de distintos proveedores) con el fin de obtener un gran sistema de base de datos (obviamente con software para poder producir la comunicación).

Silvia Alicia Gómez & Leticia Irene Gómez